Busca Aproximada e Correção Ortográfica em Tempo Real

Otimização da Distância de Levenshtein: análise de pior caso, provas e demonstração empírica

João Victor de Carvalho Montenegro
Davi Studart Fontenele

FB UNI · Análise de Algoritmos · Prof. [Nome do professor] · 2026

Resumo

A distância de Levenshtein mede o número mínimo de inserções, deleções e substituições de caracteres necessárias para transformar uma palavra em outra. É a base de corretores ortográficos e de sistemas de busca aproximada (*fuzzy search*). Este trabalho analisa três algoritmos para o problema: a recursão ingênua, o algoritmo de Wagner-Fischer (programação dinâmica) e a variante de Ukkonen com faixa diagonal limitada por um limiar de erros k. Para cada um, provamos formalmente o custo de **pior caso** e o confirmamos empiricamente contando operações numa implementação em TypeScript. Os resultados são: a recursão ingênua é exponencial, com $\Omega(4^n/n) \le T(n) \le O(9^n)$ e, de forma exata, $T(n) = \Theta((3+2\sqrt{2})^n/\sqrt{n})$; Wagner-Fischer é $\Theta(m \cdot n)$ em tempo, com espaço reduzível de $\Theta(m \cdot n)$ para $\Theta(\min(m,n))$; e Ukkonen é $\Theta(k \cdot \min(m,n))$ no pior caso e $\Omega(1)$ no melhor. Uma aplicação web interativa (Next.js + React) acompanha o trabalho e permite reproduzir cada resultado.

Palavras-chave: distância de edição; programação dinâmica; análise assintótica; pior caso; busca aproximada.

1. Introdução

Quando um usuário digita “algorimto”, o sistema precisa encontrar “algoritmo” entre milhares de palavras de um dicionário em poucos milissegundos. O critério de semelhança mais usado é a **distância de edição de Levenshtein** [1]. Como a função é avaliada milhares de vezes por consulta, o custo assintótico do algoritmo decide se a correção acontece em tempo real ou se trava o sistema.

O objetivo deste trabalho é comparar três estratégias para o mesmo problema e **provar o custo de pior caso de cada uma**, confirmando as provas com medições:

1. **Recursão ingênua** (divisão e conquista sem memoização);
2. **Wagner-Fischer** [2] (programação dinâmica *bottom-up*), incluindo a otimização de espaço para duas linhas;
3. **Ukkonen** [3] (programação dinâmica restrita a uma faixa diagonal de largura $2k+1$, com corte por limiar).

2. Definição formal do problema

Sejam $a = a_1 a_2 \cdots a_m$ e $b = b_1 b_2 \cdots b_n$ duas cadeias sobre um alfabeto Σ. Uma **operação de edição** é uma inserção, uma deleção ou uma substituição de um caractere, cada uma com custo 1. A distância $\mathrm{lev}(a, b)$ é o menor número de operações que transforma a em b.

Seja $D[i][j] = \mathrm{lev}(a_1 \cdots a_i,\ b_1 \cdots b_j)$. Então:

$$D[i][0] = i, \qquad D[0][j] = j,$$

$$D[i][j] = \min \begin{cases} D[i-1][j] + 1 & \text{(deleção de } a_i\text{)} \\ D[i][j-1] + 1 & \text{(inserção de } b_j\text{)} \\ D[i-1][j-1] + [a_i \neq b_j] & \text{(substituição ou } \textit{match}\text{)} \end{cases}$$

onde $[a_i \neq b_j]$ vale 1 se os caracteres diferem e 0 caso contrário. A resposta é $D[m][n]$.

Justificativa da recorrência (subestrutura ótima). Considere um alinhamento ótimo entre $a_1 \cdots a_i$ e $b_1 \cdots b_j$ e olhe para sua última coluna. Ela tem exatamente uma de três formas: a_i alinhado com nada (deleção), nada alinhado com b_j (inserção), ou a_i alinhado com b_j (substituição ou *match*). Removendo essa coluna, o que sobra é um alinhamento ótimo do subproblema correspondente; se não fosse, trocá-lo por um melhor reduziria o custo total, o que contradiz a otimalidade. Logo $D[i][j]$ é o mínimo das três opções.

Exemplo: $\mathrm{lev}(\texttt{kitten}, \texttt{sitting}) = 3$ (k → s, e → i, inserir g).

2.1 Modelo de custo

Adotamos o modelo RAM: comparação de caracteres, soma e mínimo de inteiros custam $O(1)$. A **operação elementar** contada é:

- na recursão ingênua, **uma chamada** da função $\mathrm{lev}(i, j)$;
- nos algoritmos de programação dinâmica, **o cálculo de uma célula** $D[i][j]$ com $i, j \geq 1$.

Cada uma executa um número constante de instruções, então o tempo total é proporcional à contagem. O **pior caso** $T(m, n)$ é o maior número de operações elementares sobre todas as entradas de tamanhos m e n. Usamos n para o caso $m = n$.

3. Abordagem 1: Recursão ingênua

```
LEV(a, b, i, j):
    se i = 0: retorne j
    se j = 0: retorne i
    c ← 0 se a[i] = b[j], senão 1
    retorne min( LEV(a, b, i-1, j) + 1,
                 LEV(a, b, i, j-1) + 1,
                 LEV(a, b, i-1, j-1) + c )
```

Toda chamada com $i, j \geq 1$ gera **sempre** três chamadas, independentemente do conteúdo das cadeias. Portanto o número de chamadas depende só de (m, n): **o pior caso, o melhor caso e o caso médio coincidem**, e qualquer entrada serve para medir o pior caso. Contando a própria chamada:

$$T(i, 0) = T(0, j) = 1, \qquad T(m, n) = 1 + T(m-1, n) + T(m, n-1) + T(m-1, n-1).$$

Teorema 1 (limite superior). $T(m, n) \leq 3^{m+n}$ para todo $m, n \geq 0$. Logo $T(n, n) = O(9^n)$.

Prova (indução em $m + n$). Base: $T(i, 0) = 1 \leq 3^i$ e $T(0, j) = 1 \leq 3^j$. Passo: sejam $m, n \geq 1$ (logo $m + n \geq 2$) e suponha a hipótese válida para somas menores. Então

$$T(m, n) \leq 1 + 3^{m+n-1} + 3^{m+n-1} + 3^{m+n-2} = 1 + \tfrac{7}{9}\, 3^{m+n} \leq 3^{m+n},$$

pois a última desigualdade equivale a $3^{m+n} \geq 9/2$, verdadeiro para $m + n \geq 2$. ■

Teorema 2 (limite inferior). $T(m, n) \geq \binom{m+n}{m}$. Em particular, $T(n, n) \geq \dfrac{4^n}{2n+1} = \Omega(4^n/n)$.

Prova. Defina $L(m, n) = \binom{m+n}{m}$. Temos $L(i, 0) = L(0, j) = 1$ e, pela regra de Pascal, $L(m, n) = L(m-1, n) + L(m, n-1)$. Descartando termos não negativos da recorrência de T, obtemos $T(m, n) \geq T(m-1, n) + T(m, n-1)$, com as mesmas bases. Por indução em $m + n$, $T(m, n) \geq L(m, n)$. Para $m = n$: $\binom{2n}{n}$ é o maior dos $2n + 1$ termos de $\sum_{t=0}^{2n} \binom{2n}{t} = 4^n$, logo $\binom{2n}{n} \geq 4^n/(2n+1)$. ■

Conclusão. $\Omega(4^n/n) \leq T(n, n) \leq O(9^n)$: o algoritmo é **exponencial** no pior caso (e em todos os casos).

Observação (valor exato). Fazendo $U = T + \frac{1}{2}$, a recorrência vira $U(m, n) = U(m-1, n) + U(m, n-1) + U(m-1, n-1)$ com $U = \frac{3}{2}$ nas bordas. Essa é a recorrência dos **números de Delannoy** $\mathscr{D}(m, n)$ [6], logo

$$T(m, n) = \frac{3\,\mathscr{D}(m, n) - 1}{2}, \qquad \mathscr{D}(n, n) = \Theta\!\left(\frac{(3 + 2\sqrt{2})^n}{\sqrt{n}}\right),$$

e portanto $T(n, n) = \Theta((1 + \sqrt{2})^{2n}/\sqrt{n}) \approx \Theta(5{,}83^n/\sqrt{n})$. Verificação: $\mathscr{D}(2, 2) = 13$ dá $T(2, 2) = 19$ e $\mathscr{D}(3, 3) = 63$ dá $T(3, 3) = 94$, valores que coincidem com a contagem medida (Tabela 1).

> **Nota.** O limite $O(3^{\min(m,n)})$, às vezes citado para este algoritmo, não é um limite superior válido: 3^n é na verdade um limite *inferior* (basta seguir a diagonal). A Tabela 1 mostra $T(13,13) \approx 2,1 \cdot 10^9$, enquanto $3^{13} \approx 1,6 \cdot 10^6$.

Espaço. A profundidade máxima da pilha é $m + n$ (cada chamada reduz $i + j$ em pelo menos 1), logo o espaço é $O(m + n)$.

4. Abordagem 2: Wagner-Fischer (programação dinâmica)

A recursão recalcula os mesmos subproblemas inúmeras vezes, mas só existem $(m + 1)(n + 1)$ subproblemas distintos. Wagner e Fischer [2] preenchem a tabela D de baixo para cima, linha a linha. Quando $D[i][j]$ é calculado, as três dependências já estão prontas.

```
WAGNER-FISCHER(a, b):
    para i ← 0 até m: D[i][0] ← i
    para j ← 0 até n: D[0][j] ← j
    para i ← 1 até m:
        para j ← 1 até n:
            c ← 0 se a[i] = b[j], senão 1
            D[i][j] ← min(D[i-1][j] + 1, D[i][j-1] + 1, D[i-1][j-1] + c)
    retorne D[m][n]
```

Correção. Por indução na ordem de preenchimento (linha a linha, da esquerda para a direita): ao calcular $D[i][j]$, as células $(i-1, j)$, $(i, j-1)$ e $(i-1, j-1)$ vêm antes nessa ordem e já contêm os valores corretos. A atribuição aplica exatamente a recorrência da Seção 2.

Teorema 3 (tempo). O algoritmo executa $\Theta(m \cdot n)$ operações para **qualquer** entrada; o pior caso é $\Theta(m \cdot n)$, ou $\Theta(n^2)$ quando $m = n$.

Prova. Sejam c_1, c_2, c_3 as constantes de custo do corpo do laço interno, de uma iteração de inicialização e do trecho fixo. Então

$$T(m,n) = c_3 + c_2\,(m + n + 1) + \sum_{i=1}^{m} \sum_{j=1}^{n} c_1 = c_1\,mn + c_2\,(m + n + 1) + c_3\,.$$

Para $m, n \geq 1$, vale $m + n + 1 \leq 3mn$, logo $c_1\,mn \leq T(m,n) \leq (c_1 + 3c_2 + c_3)\,mn$, isto é, $T = \Theta(mn)$. O número de iterações não depende dos caracteres, então todas as entradas de tamanho (m, n) custam o mesmo. ■

Teorema 4 (espaço). A tabela completa ocupa $(m + 1)(n + 1) = \Theta(mn)$ células. Mantendo apenas a linha anterior e a atual, e escolhendo como colunas a cadeia mais curta, o espaço cai para $2(\min(m,n) + 1) = \Theta(\min(m,n))$, sem alterar o tempo.

Prova. O cálculo da linha i lê apenas a linha $i-1$ (dependências de cima e da diagonal) e a própria linha i (dependência da esquerda). Depois que a linha i está completa, nenhuma linha anterior a i volta a ser lida e pode ser descartada. Como $\mathrm{lev}(a,b) = \mathrm{lev}(b,a)$, podemos trocar as cadeias para que o comprimento das linhas seja $\min(m,n)$. ■

O custo dessa otimização é perder a tabela usada no *backtracking*, que reconstrói o alinhamento. Para obter o alinhamento em espaço linear existe o algoritmo de Hirschberg [4]. A Tabela 3 mostra o impacto prático: para $m = n = 10^5$ com inteiros de 4 bytes, a tabela completa precisaria de cerca de 37 GB, enquanto as duas linhas ocupam 781 KB.

5. Abordagem 3: Ukkonen (faixa diagonal com limiar k)

Num corretor ortográfico só interessam palavras **próximas**: queremos saber se $\mathrm{lev}(a,b) \le k$ para um limiar pequeno (tipicamente $k \le 3$). Ukkonen [3] observou que, nesse caso, só uma faixa diagonal da tabela precisa ser calculada.

```
UKKONEN(a, b, k):                          ▷ supõe m ≥ n (troca se preciso)
    se m − n > k: retorne "> k"            ▷ corte em O(1)
    INF ← k + 1
    para i ← 1 até m:
        para j ← max(1, i−k) até min(n, i+k):
            V[i][j] ← min(V[i−1][j] + 1, V[i][j−1] + 1,
                          V[i−1][j−1] + [a_i ≠ b_j], INF)
        se min_j V[i][j] > k: retorne "> k"   ▷ corte antecipado
    retorne V[m][n] se V[m][n] ≤ k, senão "> k"
```

Células fora da faixa $|i-j| \le k$ valem INF $= k+1$. Na implementação, apenas duas linhas são mantidas, como na Seção 4.

Lema 1. $D[i][j] \ge |i-j|$.

Prova. Cada operação de edição altera o comprimento da cadeia em no máximo 1. Para transformar uma cadeia de comprimento i numa de comprimento j são necessárias pelo menos $|i-j|$ operações. ■

Lema 2 (valores truncados). Para toda célula da faixa, $V[i][j] = \min(D[i][j],\ k+1)$.

Prova (indução na ordem de preenchimento). Pelo Lema 1, toda célula fora da faixa tem $D > k$, e portanto $(D, k+1) = k+1 =$ INF, que é exatamente o valor que o algoritmo lhe atribui. Para uma célula da faixa, suponha que as três vizinhas valem $\min(D_*, k+1)$. Como somar uma constante não negativa e tomar mínimos comuta com o truncamento em $k+1$, temos $\min(\min(x, k+1) + w,\ \ldots,\ k+1) = \min(\min(x+w, \ldots),\ k+1)$. Logo $V[i][j] = \min(D[i][j], k+1)$. ■

Lema 3 (corte antecipado). Se, para alguma linha i, $\min_j V[i][j] > k$, então $D[m][n] > k$.

Prova. Suponha, por contradição, que $d = D[m][n] \le k$. O *backtracking* a partir de (m, n) define um caminho ótimo até $(0, 0)$ em que os valores de D não diminuem no sentido de $(0, 0)$ para (m, n), pois cada passo soma um custo 0 ou 1. Esse caminho atravessa a linha i em alguma célula (i, j) com $D[i][j] \le d \le k$. Pelo Lema 1, $|i - j| \le k$, então a célula está na faixa e, pelo Lema 2, $V[i][j] = D[i][j] \le k$. Isso contradiz a hipótese. ■

Teorema 5 (correção). O algoritmo devolve lev(a, b) se este valor é $\le k$, e "> k" caso contrário. *Prova:* o corte por comprimento é correto pelo Lema 1 aplicado a (m, n); o corte antecipado pelo Lema 3; e o valor final pelo Lema 2. ■

Teorema 6 (pior caso). Com $n = \min(m, n)$ e $k \ge 1$, o algoritmo executa $O(k \cdot n)$ operações. Esse limite é atingido: para $a = b$ com $|a| = n \ge k$, o número exato de células calculadas é

$$T = (2k + 1)\,n - k(k + 1),$$

que é $\ge (k + 1)\,n$ para $n > k$. Portanto o pior caso é $\Theta(k \cdot \min(m, n))$.

Prova. **(Limite superior.)** Se $m - n > k$, o algoritmo termina em $O(1)$. Caso contrário $m \le n + k$, e cada linha calcula no máximo $\min(n, 2k + 1)$ células. Assim $T \le m \cdot \min(n, 2k + 1)$. Se $k \le n$, temos $T \le (n + k)(2k + 1) \le 2n \cdot 3k = 6kn$. Se $k > n$, temos $T \le (n + k)\,n < 2kn$. Em ambos os casos, $T = O(k \cdot n)$.

(O limite é atingido.) Se $a = b$, então $D[i][i] = 0$ em toda a diagonal. O mínimo de cada linha é 0 e o corte antecipado nunca ocorre, então toda a faixa é calculada. As células internas fora da faixa satisfazem $|i - j| > k$. Do lado $j > i$ há $\sum_{d=k+1}^{n-1} (n - d) = \frac{(n-k)(n-k-1)}{2}$ delas, e o mesmo número do outro lado. Logo

$$T = n^2 - (n - k)(n - k - 1) = (2k + 1)\,n - k(k + 1).$$

Por fim, $T - (k + 1)n = k(n - k - 1) \ge 0$ para $n > k$, ou seja, $T = \Omega(k \cdot n)$. ■

Melhor caso. Se $|m - n| > k$, a resposta sai em $O(1)$ sem calcular nenhuma célula: $\Omega(1)$. Com entradas aleatórias, o corte antecipado também costuma ocorrer nas primeiras linhas. Nas medições da Tabela 2, cadeias aleatórias com $k = 5$ calcularam entre 51 e 84 células, independentemente de n.

Espaço. Com duas linhas, $\Theta(\min(m, n))$, como no Teorema 4.

6. Validação empírica

Todos os algoritmos foram implementados em TypeScript (arquivo `src/lib/levenshtein.ts`) com um **contador de operações elementares**. A correção foi verificada por testes automáticos (`npm run check`):

- 5 000 pares aleatórios, nos quais as quatro implementações (recursiva, matriz completa, duas linhas e Ukkonen com $k = 0..12$) concordam;

- o contador da recursão é **igual**, chamada a chamada, ao valor previsto pela recorrência $T(m, n)$;
- a fórmula exata $(2k + 1)n - k(k + 1)$ do Teorema 6 vale para todo $1 \le n \le 60$ e $0 \le k \le n$.

Os tempos abaixo foram medidos com Node.js 22 (motor V8) num Intel Xeon 2,1 GHz. No navegador os valores absolutos variam, mas as proporções se mantêm.

Tabela 1 — Recursão ingênua (qualquer entrada). Contagem medida igual à prevista em todas as linhas.

n	**Chamadas** $T(n, n)$	$T(n)/T(n-1)$	**Tempo**
4	481	5,117	0,03 ms
6	13 483	5,342	0,62 ms
8	398 593	5,463	3,4 ms
9	2 193 844	5,504	15,9 ms
10	12 146 179	5,536	79,5 ms
11	67 570 078	5,563	434 ms
12	377 393 953	5,585	**2,41 s**
13	$2,1 \cdot 10^9$ (previsto)	5,604	≈ 13 s (extrapolado)
20	$3,9 \cdot 10^{14}$ (previsto)	—	≈ 29 dias
30	$1,4 \cdot 10^{22}$ (previsto)	—	≈ 2,9 milhões de anos

A razão $T(n)/T(n-1)$ fica entre 4 e 9 (Teoremas 1 e 2) e converge para $3 + 2\sqrt{2} \approx 5,83$ (Observação da Seção 3). Cada letra a mais multiplica o tempo por cerca de 5,6: com 12 letras a função já leva segundos, o que confirma empiricamente o limite de 12–13 letras.

Tabela 2 — Wagner-Fischer (duas linhas) e Ukkonen ($k = 5$). No Ukkonen foi usado o pior caso ($a = b$).

n	**WF: células**	**WF:** ops/n^2	**WF: tempo**	**Ukkonen: células**	**Ukkonen:** ops/$((2k + 1)n)$	**Ukkonen: tempo**	**Ukkonen (aleatória): células**
100	10 000	1,000	0,06 ms	1 070	0,973	0,010 ms	62
500	250 000	1,000	1,8 ms	5 470	0,995	0,039 ms	84
1 000	1 000 000	1,000	7,3 ms	10 970	0,997	0,077 ms	62
2 000	4 000 000	1,000	29,7 ms	21 970	0,999	0,159 ms	51
4 000	16 000 000	1,000	96,1 ms	43 970	0,999	0,289 ms	62

Leitura da tabela:

- **Wagner-Fischer.** A razão ops/n^2 é exatamente 1 e o tempo quadruplica quando n dobra: comportamento quadrático.
- **Ukkonen.** A razão ops/$((2k+1)n)$ converge para 1 e o tempo dobra quando n dobra: comportamento linear em n para k fixo.
- **Curvas planas.** Uma razão ops/limite que fica constante quando n cresce é a evidência empírica de que o limite de pior caso provado é apertado.

> **Lição de implementação.** Uma primeira versão do Ukkonen reiniciava a linha inteira (`fill`) a cada iteração. O contador de células continuava linear, mas o tempo crescia de forma quadrática, porque o `fill` custa $O(n)$ por linha. A versão final reinicia apenas a célula de borda da faixa. Isso mostra que a operação elementar escolhida na análise precisa de fato dominar o custo real.

Tabela 3 — Memória (inteiros de 4 bytes, $m = n$).

$m = n$	Matriz completa	Duas linhas	Economia
100	39,8 KB	808 B	98,02%
1 000	3,82 MB	7,82 KB	99,80%
10 000	382 MB	78,1 KB	99,98%
100 000	37,3 GB	781 KB	99,998%

7. Análise comparativa

	Recursão ingênua	Wagner-Fischer	Ukkonen
Pior caso (tempo)	$\Theta((3+2\sqrt{2})^n/\sqrt{n})$	$\Theta(m \cdot n)$	$\Theta(k \cdot \min(m,n))$
Melhor caso (tempo)	igual ao pior	igual ao pior	$\Omega(1)$
Espaço	$O(m+n)$ (pilha)	$\Theta(mn)$ ou $\Theta(\min(m,n))$	$\Theta(\min(m,n))$
Resposta	distância exata	distância exata + alinhamento	distância se $\leq k$
Técnica de prova	indução + Pascal	somatório duplo	lema da faixa + contagem exata

Para k fixo e $n \to \infty$, a hierarquia é $\Theta(k\,n) \ll \Theta(n^2) \ll \Theta(5,83^n/\sqrt{n})$. Num corretor ortográfico que compara a palavra digitada com N palavras do dicionário, o custo total passa de $N \cdot \Theta(n^2)$ para $N \cdot O(k\,n)$. Na prática é ainda menor, porque a maioria das palavras do dicionário é descartada pelo corte de comprimento ou pelo corte antecipado.

8. Aplicação interativa (Levenshtein Lab)

Como demonstração prática, foi desenvolvida uma aplicação web em **Next.js 15, React 19, TypeScript e Tailwind CSS 4**, com tema escuro e layout em tela dividida: controles à esquerda e visualização à direita. Os algoritmos ficam numa biblioteca pura (`src/lib/levenshtein.ts`), sem dependência de React, que é compartilhada pela interface e pelos testes.

Módulo	Função	Resultado do relatório que demonstra
A. Simulador da matriz	Preenche D célula a célula (play/pause, avançar, retroceder), colore cada operação e desenha o caminho ótimo do *backtracking*	Recorrência (Seção 2) e Teorema 3
B. Stress test	Mede operações × n ao vivo num *Web Worker*; mostra a razão $T(n)/T(n-1)$ e as razões ops/limite	Teoremas 1, 2, 3 e 6; Tabelas 1 e 2
C. Árvore de recorrência	Desenha a árvore de chamadas da recursão, com subproblemas repetidos em vermelho	Desperdício da Seção 3 e motivação da PD
D. Corte de Ukkonen	*Slider* de k que aplica a máscara diagonal; mostra células calculadas, ignoradas e abandonadas	Lemas 1 a 3 e Teorema 6
E. Raio-X de memória	Compara a matriz completa com duas linhas, até $n = 10^6$	Teorema 4 e Tabela 3

Decisões de engenharia.

- O *benchmark* roda num *Web Worker*: mesmo quando a recursão leva segundos, a interface continua responsiva. O Módulo B tem também um botão que executa a recursão na *thread* principal, para mostrar o travamento real.
- Acima de um orçamento de chamadas, a recursão é abortada e a curva continua com a contagem **exata** prevista pela recorrência (calculada em $O(mn)$), sem executar bilhões de chamadas.
- A aplicação é exportada como site estático, sem servidor.

Para executar: `npm install && npm run dev`, depois abrir `http://localhost:3000`. Os testes rodam com `npm run check`.

9. Conclusão

Provamos formalmente o custo de pior caso das três abordagens e confirmamos cada limite com contagens exatas de operações:

- a **recursão ingênua** é exponencial em todas as entradas, crescendo cerca de $5,83\times$ por letra;

- **Wagner-Fischer** elimina a recomputação e atinge $\Theta(mn)$, com espaço reduzível a $\Theta(\min(m, n))$;
- **Ukkonen** explora o limiar k e atinge $\Theta(k \cdot \min(m, n))$ no pior caso e $\Omega(1)$ no melhor, o que o torna adequado para correção ortográfica em tempo real.

O trabalho também mostra que a análise assintótica precisa estar alinhada com a implementação: um detalhe como reiniciar uma linha inteira pode, sem alterar a contagem de células, transformar um algoritmo linear em quadrático.

Referências

[1] LEVENSHTEIN, V. I. Binary codes capable of correcting deletions, insertions, and reversals. *Soviet Physics Doklady*, v. 10, n. 8, p. 707–710, 1966.

[2] WAGNER, R. A.; FISCHER, M. J. The string-to-string correction problem. *Journal of the ACM*, v. 21, n. 1, p. 168–173, 1974.

[3] UKKONEN, E. Algorithms for approximate string matching. *Information and Control*, v. 64, n. 1–3, p. 100–118, 1985.

[4] HIRSCHBERG, D. S. A linear space algorithm for computing maximal common subsequences. *Communications of the ACM*, v. 18, n. 6, p. 341–343, 1975.

[5] CORMEN, T. H.; LEISERSON, C. E.; RIVEST, R. L.; STEIN, C. *Introduction to Algorithms*. 4. ed. Cambridge: MIT Press, 2022.

[6] BANDERIER, C.; SCHWER, S. Why Delannoy numbers? *Journal of Statistical Planning and Inference*, v. 135, n. 1, p. 40–54, 2005.

[7] NAVARRO, G. A guided tour to approximate string matching. *ACM Computing Surveys*, v. 33, n. 1, p. 31–88, 2001.